

武汉大学

《计算机系统综合设计》实践报告

RV32 五级流水线含中断处理系统实现

姓 名：刘鼎琨  
学 号：2023302121009  
专 业：计算机科学与技术  
学 院：计算机学院  
指导教师：熊小兵老师

二〇二五年 六月

# 原创性声明

本人郑重声明：所呈交的论文（设计），是本人在指导教师的指导下，严格按照学校和学院有关规定完成的。除文中已经标明引用的内容外，本论文（设计）不包含任何其他个人或集体已发表及撰写的研究成果。对本论文（设计）做出贡献的个人和集体，均已在文中以明确方式标明。本人承诺在论文（设计）工作过程中没有伪造数据等行为。若在本论文（设计）中有侵犯任何方面知识产权的行为，由本人承担相应的法律责任。

作者签名： 刘鼎琨

日期： 2025 年 6 月 29 日

## 摘 要

本实践报告以 RISC-V 指令集为基础,设计并实现了一个支持中断的 RV32 五级流水线 CPU 系统。通过单周期结构向五级流水结构的扩展,系统涵盖了指令获取、解码、执行、访存与写回等处理阶段,支持 37 条基本指令,并在此基础上实现了按键中断和计数器中断功能。在中断系统中,采用了两级处理中断的机制,并引入中断裁决器进行优先级管理和嵌套控制。在此基础上,报告进一步提出通过 Vivado 中的 Pblock 区域划分技术,实现对敏感模块的物理隔离,以降低跨时钟域通信中亚稳态出现的概率,提升系统时序稳定性。最终在 Nexys A7 FPGA 平台上完成了系统的综合、布局、实现与验证,展示了完整的软硬件协同设计流程。该设计不仅验证了所学计算机组成原理知识的可行性,也体现了在工程实践中对可靠性和优化的深入思考。

**关键词:** RISC-V; 五级流水线; 中断处理; Pblock 划分; 亚稳态

# ABSTRACT

This practical report presents the design and implementation of an RV32 five-stage pipelined CPU system based on the RISC-V instruction set architecture. The system was developed from a single-cycle CPU and extended to a full five-stage pipeline, covering instruction fetch, decode, execute, memory access, and write-back. It supports 37 base instructions and integrates both button interrupts and timer interrupts. The interrupt mechanism is based on a two-level processing strategy, with an interrupt judger handling priority scheduling and nested interrupt control. Furthermore, the report introduces a Pblock-based physical constraint technique in Vivado to localize sensitive modules and mitigate metastability risks in clock domain crossing scenarios, thereby enhancing the timing reliability of the system. The final system was synthesized and deployed on the Nexys A7 FPGA development board, demonstrating a complete workflow of hardware-software co-design. This project not only consolidates knowledge from computer architecture courses but also reflects in-depth consideration of stability and system optimization in engineering practice.

**Keywords:** RISC-V; five-stage pipeline; interrupt handling; Pblock placement; metastability

# 目 录

|                                    |           |
|------------------------------------|-----------|
| 摘要 .....                           | I         |
| ABSTRACT .....                     | II        |
| <b>1 绪论 .....</b>                  | <b>2</b>  |
| 1.1 实验目的 .....                     | 2         |
| 1.2 国内外研究现状.....                   | 2         |
| 1.3 实验环境介绍 .....                   | 3         |
| <b>2 单周期 CPU 设计方案 .....</b>        | <b>4</b>  |
| 2.1 单周期结构框架设计 .....                | 4         |
| 2.2 开发板外设使用说明及结果展示 .....           | 5         |
| <b>3 五级流水线 CPU 设计方案 .....</b>      | <b>6</b>  |
| 3.1 五级流水线结构框架设计 .....              | 6         |
| 3.2 模块定义及功能分析 .....                | 6         |
| 3.3 模块接口定义 .....                   | 10        |
| 3.4 流水线架构的进一步优化 .....              | 11        |
| <b>4 按键中断、计数器中断设计方案 .....</b>      | <b>13</b> |
| 4.1 中断方案设计 .....                   | 13        |
| 4.2 中断发生器 INT-CONTROLLER 设计 .....  | 13        |
| 4.3 中断裁决器 INT_JUDGER 设计 .....      | 14        |
| <b>5 附加功能：PBLOCK 划分避免亚稳态 .....</b> | <b>16</b> |
| 5.1 引入原因 .....                     | 16        |
| <b>6 实验测试方案及结果 .....</b>           | <b>18</b> |
| 6.1 测试方案 .....                     | 18        |
| 6.2 测试代码与结果预期 .....                | 18        |
| 6.3 测试结果与评价 .....                  | 20        |
| <b>7 结论 .....</b>                  | <b>21</b> |
| 参考文献 .....                         | 22        |
| 致 谢 .....                          | 23        |

|                   |    |
|-------------------|----|
| 附录：模块核心代码列举 ..... | 24 |
|-------------------|----|

# 1 绪论

## 1.1 实验目的

本次实验旨在通过实践深化学生对计算机体系结构的理解，重点培养其硬件设计能力。在实验过程中，学生需要基于RISC-V指令集架构，从零开始设计并实现支持37条基础指令的单周期CPU，进而扩展为流水线结构的处理器。实验特别注重中断机制的设计与实现，要求学生完成包括按键中断和计数器中断在内的多种中断处理功能。通过这一系列实践环节，不仅巩固了计算机组成原理课程的核心知识，更有效提升了学生的自主研究能力和工程实践水平，为后续计算机系统相关领域的学习和研究打下坚实基础。

## 1.2 国内外研究现状

计算机体系结构作为计算机科学的核心领域之一，其研究与发展始终受到学术界和产业界的高度关注。近年来，随着RISC-V开源指令集架构的兴起，处理器设计领域呈现出新的研究活力。国内外众多高校和研究机构纷纷将RISC-V架构引入教学实践，通过简化但完整的处理器设计实验，帮助学生深入理解计算机组成原理<sup>[1]</sup>。

在国际研究层面，以MIT、Berkeley等为代表的高校在计算机体系结构教学中普遍采用实践导向的培养模式。这些院校不仅注重理论知识的传授，更强调通过完整的处理器设计项目来培养学生的系统思维能力。特别是Berkeley开发的RISC-V教学项目，为全球计算机体系结构教学提供了重要参考。这些项目通常要求学生从最基础的逻辑门开始，逐步构建支持关键指令集的处理器核心。

国内高校在计算机组成原理教学改革方面也取得了显著进展。清华大学、北京大学等顶尖院校率先将RISC-V架构引入实验教学，开发了系列化的处理器设计实验项目。这些项目不仅覆盖单周期CPU设计等基础内容，还包括流水线优化、中断处理等进阶主题。通过这种循序渐进的教学设计，学生能够建立起对计算机系统工作原理的完整认知。值得注意的是，这些教学实践都特别强调对学生创新能力和工程实践能力的培养<sup>[2-3]</sup>。

### 1.3 实验环境介绍

**ModelSim:** 使用 ModelSim 进行 Verilog 代码的功能仿真，检查时序逻辑和组合逻辑的正确性。通过波形分析验证了 CPU 各模块的协同工作，确保设计符合 RISC-V 指令规范。

**Verilog HDL:** 在本项目中，Verilog HDL 用于描述 CPU 的硬件结构，包括数据通路、控制单元和中断逻辑。通过编写模块化的 Verilog 代码实现了单周期和流水线 CPU 的核心功能，并进行仿真验证。

**Venus:** Venus 作为 RISC-V 模拟器，用于验证汇编程序的正确性，确保指令执行符合预期。在调试阶段，可以利用 Venus 对比 CPU 的行为，排查指令执行中的逻辑错误。

**Vivado:** Vivado 用于综合和实现 CPU 设计，生成可在 FPGA 上运行的比特流文件。借助其调试工具，通过分析了关键路径的时序性能，优化了流水线的执行效率。

**Nexys A7:** 该开发板作为硬件验证平台，运行设计的 SCPU，并通过按键和计数器中断测试其实时响应能力。最终成功在 FPGA 上验证了 37 条指令的正确执行。

在实验中，额外添加了七段显示器模块 (seg7\_display)，用于将 CPU 的当前状态信息以直观的方式显示，实验过程中可以通过拨动开关查看不同部件的运行状态。

## 2.2 开发板外设使用说明及结果展示

拨码开关的定义与利用：定义 sw\_i[15] 切换运行速度，sw\_i[0] 控制指令暂停调试，sw\_i[1] 控制暂停查看 RF，sw\_i[2] 控制 ALU 结果暂停，sw\_i[3] 控制 DM 结果暂停，sw\_i[10:4] 作为 DM 地址选择信号，sw\_i[14] 用于 ROM 显示，sw\_i[13] 用于 RF 显示，sw\_i[12] 用于 ALU 显示，sw\_i[11] 用于 DM 显示。

效果上仅展示几种典型模式，其余可以参考文件夹中的演示视频：

- 展示 1：程序默认运行，不显示



- 展示 2：动态运行模式下显示 ROM 程序代码



图2.2 单周期架构演示截图（部分）

单周期 CPU 的设计为后续流水线的引入提供了重要支持和思想方法的规范，也让我体会到在实现具体代码之前先设计好模块接口的重要性，能大幅度提升编程的效率。

## 3 五级流水线 CPU 设计方案

### 3.1 五级流水线结构框架设计

根据教材<sup>[4]</sup>对五级流水线的规范(如图 3.1 所示),相较于单周期 CPU 应引入流水线寄存器,以及其他用于处理数据、控制冒险的模块。

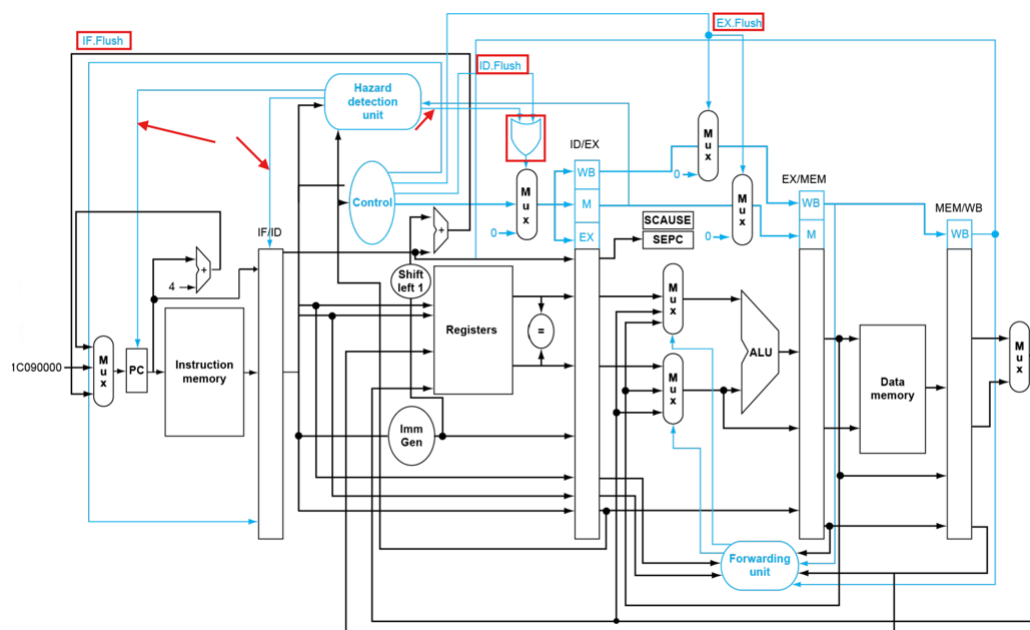


图3.1 五级流水线CPU结构框架

针对教材的经典设计进行了有针对性的改进,引入了 NPC 模块用于确定下一条指令的确定值,保留了单周期设计中 Branch 和 Zero 信号规约的与门,取缔了专用的 ALU control 模块。

根据一个模块的 input 信号在流水线中最靠后的位置,可以把模块进行划分: IF 阶段包括地址变换 PC 模块; ID 阶段包括控制信号生成 Ctrl1 模块、立即数拓展 EXT 模块、寄存器 RF 模块; EX 阶段包括旁路的处理模块(即上图中并列的两个 MUX 模块)、冒险检测单元 Hazzard Detecting Unit、ALU 模块、停顿单元 Stall Unit、NPC 模块; MEM 阶段包括数据存储器 DM、前递单元 Forwarding Unit; WB 阶段包括 MUX 选择模块、RF 模块(在时钟下降沿触发,和 ID 阶段的 RF 模块不冲突)。

### 3.2 模块定义及功能分析

#### 3.2.1 IF 阶段

IF (Instruction Fetch) 阶段主要负责指令的预取工作。PC 模块作为核心控制单元，在每个时钟上升沿更新程序计数器值，并将当前 PC 指向的指令送入 IF\_ID 流水线寄存器。该阶段实现了基础的指令流水线机制，为后续处理环节提供稳定的指令流。值得注意的是，在支持中断的扩展设计中，PC 模块还需要处理异常跳转地址的计算和保存，确保中断返回时能恢复原有执行流程。

### 3.2.2 ID 阶段

ID (Instruction Decode) 阶段完成指令解码和寄存器读取工作。控制信号生成模块采用三级译码机制：首先根据操作码 (opcode) 确定指令大类，再结合 func3 和 func7 字段细化具体操作类型，最终生成包括 ALU 控制、存储器访问、寄存器写入等在内的完整控制信号集。RegWrite 用于 RF 模块，MemWrite、Memread、dm\_ctrl 用于 dm\_controller 模块，ALUSrc、ALUOp 用于 ALU，EXTOp 用于 EXT 模块，WDSel (教材称为 MemtoReg 信号) 用于 MUX 模块，NPCOp 用于 NPC 模块；

立即数扩展模块支持多种格式的立即数处理，包括 I 型、S 型、B 型等 RISC-V 标准编码格式，为后续运算提供正确的操作数，遵循的规则如图 3.2 所示。

| Name<br>(Field Size) | 7 bits                      | 5 bits | 5 bits | 3 bits | 5 bits        | 7 bits | Comments                      |
|----------------------|-----------------------------|--------|--------|--------|---------------|--------|-------------------------------|
| R-type               | funct7                      | rs2    | rs1    | funct3 | rd            | opcode | Arithmetic instruction format |
| I-type               | immediate[11:0]             |        | rs1    | funct3 | rd            | opcode | Loads & immediate arithmetic  |
| S-type               | immed[11:5]                 | rs2    | rs1    | funct3 | immed[4:0]    | opcode | Stores                        |
| SB-type              | immed[12,10:5]              | rs2    | rs1    | funct3 | immed[4:1,11] | opcode | Conditional branch format     |
| UJ-type              | immediate[20,10:1,11,19:12] |        |        |        | rd            | opcode | Unconditional jump format     |
| U-type               | immediate[31:12]            |        |        |        | rd            | opcode | Upper immediate format        |

图3.2 Risc-v32的指令类型及分段

寄存器模块 RF 通过 IF\_ID\_rs1、IF\_ID\_rs2 读取寄存器的值，放入 ID\_EX 寄存器。采用异步读取、同步写入的设计，通过前递逻辑有效解决了部分数据冒险问题，核心的代码如下：

```
assign RD1 = (A1 != o) ? ((RfWr && A3 == A1) ? WD : rf[A1]) : o;
assign RD2 = (A2 != o) ? ((RfWr && A3 == A2) ? WD : rf[A2]) : o;
```

### 3.2.3 EX 阶段

EX (Execution) 阶段是数据处理的核心环节。ALU 模块支持算术运算、逻

辑运算、移位操作等基本运算，并根据控制信号动态选择运算类型。旁路处理单元会根据 ForwardA 和 ForwardB 两个由 Forward 模块生成的信号，来选择更新 RD1 和 RD2。Hazzard Detecting Unit 能够有效处理 Load-Use 型数据冒险，通过插入气泡周期保证数据正确性(如图 3.3 所示)。而特别设计的控制冒险 Stall 模块通过 ALU 模块产生的 Zero 信号，去更新 NPCOp(如图 3.4 所示)。NPC 模块用于确定下一跳指令的 PC 值，主要是根据 ctrl 模块中的 NPCOp 信号确定下一个地址的位置。

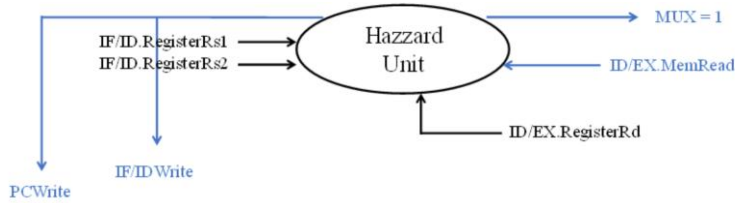


图3.3 Hazzard Detecting Unit输入输出接口图

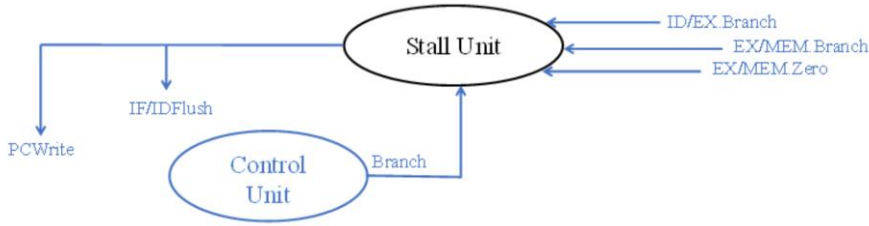


图3.4 Stall Unit输入输出接口图

### 3.2.4 MEM 阶段

MEM (Memory Access) 阶段负责数据存储器的读写操作。数据存储器采用 Vivado 提供的 Block Memory Generator IP 核实现，支持单周期读写操作。前递单元 Forwarding Unit 在此阶段完成最终的数据前递判断，确保写入寄存器的数据是最新结果(如图 3.5 所示)。在扩展设计中，该阶段还集成了中断响应机制，能够保存关键寄存器状态并跳转到中断服务例程。存储器访问控制信号经过精心设计，确保与 RISC-V 存储模型完全兼容。

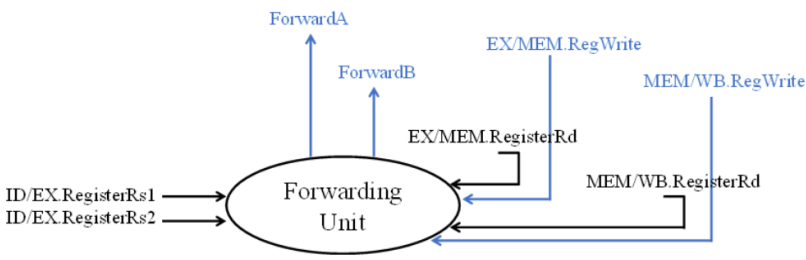


图3.5 Forwarding Unit输入输出接口图

### 3.2.5 WB 阶段

WB (Write Back) 阶段完成运算结果的写回操作。多路选择器 MUX 根据控制信号选择正确的写回数据源, 支持 ALU 运算结果、存储器读取数据、PC 三种写回路径。寄存器模块 RF 采用时钟下降沿触发的写入机制, 避免了与指令读取的时序冲突。在支持中断的扩展实现中, 该阶段还负责中断返回时的现场恢复工作, 确保程序能够从中断点继续正确执行。整个写回过程严格遵循 RISC-V 的寄存器更新语义, 保证体系结构状态的正确性。

### 3.2.6 流水线寄存器设计

在五级流水线中, 每个流水线寄存器应该保存不同的信息, 因为要将其前一个阶段的处理结果进行保存, 包括一些晚期用到的控制信号也要保证留存。就如 Ctrl 模块位于 ID 阶段, 则 IF/ID 寄存器应该存储 Ctrl 模块需要的指令信息, 而 ID/EX 寄存器应该存储后续阶段的相关控制信号, 如 Mem\_read 等。

IF-ID 寄存器保存了: int\_ret, PC\_out, rs1, rs2, rd, iimm\_shamt, iimm, simm, bimm, uimm, jimm, Op, Funct7, Funct3;

ID-EX 寄存器保存了: IF\_ID\_int\_ret, IF\_ID\_rd, IF\_ID\_rs1, IF\_ID\_rs2, RegWrite, Memread, Memwrite, EXTOp, ALUOp, NPCOp, ALUSrc, WDSel, dm\_c, immout, RD1, RD2, IF\_ID\_PC\_out;

EX-MEM 寄存器保存了: ID\_EX\_int\_ret, aluout, ID\_EX\_PC\_out, ID\_EX\_WDSel, ID\_EX\_RegWrite, ID\_EX\_Memread, ID\_EX\_Memwrite, ID\_EX\_NPCOp, ID\_EX\_dm\_c, ID\_EX\_rd, B\_in;

MEM-WB 寄存器保存了: EX\_MEM\_int\_ret, Data\_in, EX\_MEM\_RegWrite, EX\_MEM\_rd, EX\_MEM\_WDSel, EX\_MEM\_aluout, EX\_MEM\_PC\_out。

流水线寄存器的处理可以统一使用一个模块, 因为每个流水线寄存器的功能是一致的, 即保存、清除 (Flush)、暂停 (write\_enable)、保护与恢复。因此本项目中将流水线寄存器共用模块, 分别实例化。

### 3.3 模块接口定义

该项目中定义了诸多模块的接口，为了清晰表达结构设计，在此陈列核心模块的定义接口(如表 3.1~3.5 所示)。

表 3.1 DataForwardUnit 接口定义

| 信号名           | 方向     | 描述               |
|---------------|--------|------------------|
| exMemRegWrite | input  | EX/MEM 阶段寄存器写使能  |
| exMemRd       | input  | EX/MEM 阶段写寄存器号   |
| memWbRegWrite | input  | MEM/WB 阶段寄存器写使能  |
| memWbRd       | input  | MEM/WB 阶段写寄存器号   |
| idExRs1       | input  | ID/EX 阶段源寄存器 1 号 |
| idExRs2       | input  | ID/EX 阶段源寄存器 2 号 |
| forwardA      | output | 源操作数 A 旁路选择信号    |
| forwardB      | output | 源操作数 B 旁路选择信号    |

表 3.2 PC 接口定义

| 信号名   | 方向     | 描述        |
|-------|--------|-----------|
| clkIn | input  | 时钟信号      |
| rstN  | input  | 复位信号（低有效） |
| npcIn | input  | 下一条指令地址   |
| pcOut | output | 当前 PC 值   |

表 3.3 Ctrl

| 信号名      | 方向     | 描述           |
|----------|--------|--------------|
| Op       | input  | 指令操作码        |
| Funct7   | input  | 指令 funct7 字段 |
| Funct3   | input  | 指令 funct3 字段 |
| RegWrite | output | 寄存器写使能       |
| MemWrite | output | 存储器写使能       |
| EXTOp    | output | 立即数扩展控制      |
| ALUOp    | output | ALU 操作码      |
| NPCOp    | output | 下一 PC 操作码    |
| ALUSrc   | output | ALU 源选择      |
| GPRSel   | output | 通用寄存器选择      |
| WDSel    | output | 写数据选择        |
| DMType   | output | 数据存储类型       |
| dm_ctrl  | output | 数据存储器控制信号    |
| Memread  | output | 存储器读使能       |

表 3.4 MemAccessCtrl

| 信号名        | 方向    | 描述     |
|------------|-------|--------|
| memWriteEn | input | 存储器写使能 |
| addrIn     | input | 地址输入   |

|                 |        |           |
|-----------------|--------|-----------|
| dataWrite       | input  | 写入数据      |
| dmCtrl          | input  | 数据存储器控制信号 |
| dataReadRaw     | input  | 原始读出数据    |
| dataRead        | output | 处理后的读出数据  |
| dataWriteOut    | output | 处理后的写入数据  |
| writeEnableMask | output | 写使能掩码     |

表 3.5 clk\_div

| 信号名      | 方向     | 描述        |
|----------|--------|-----------|
| clk_in   | input  | 时钟输入      |
| rst_n    | input  | 复位信号（低有效） |
| sel_sw   | input  | 选择开关      |
| div_cnt  | output | 分频计数输出    |
| cpu_clk  | output | CPU 时钟输出  |
| int_flag | output | 中断标志输出    |

### 3.4 流水线架构的进一步优化

**分支预测机制增强：**原始 IF 阶段存在分支判定延迟问题。改进方案建议引入多级预测架构(如图 3.6 所示)：采用静态“恒不跳转”策略维持基础指令流连续性，结合动态分支目标缓冲（BTB）机制记录历史跳转轨迹。当程序计数器命中 BTB 时直接预取目标地址指令，理论上可将预测准确率提升至 85%以上。该设计可减少约 60%的控制冒险停顿周期<sup>[5]</sup>。

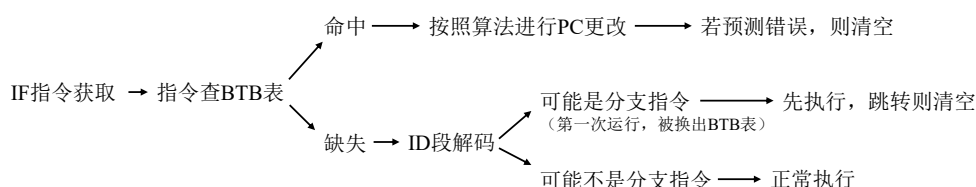


图3.6 指令局部预测的实现思路

**分支判定时序优化：**为降低分支指令延迟，拟将判定逻辑迁移至 ID 阶段(如图 3.6 所示)。通过在指令解码环节同步识别 B/J-type 指令，配合专用比较电路提前生成 Zero 标志位，同时并行计算跳转目标地址，使分支判定周期从 EX 阶段提前至 ID 阶段结束前。理论分析表明，此举可减少 2 个时钟周期的流水线气泡。

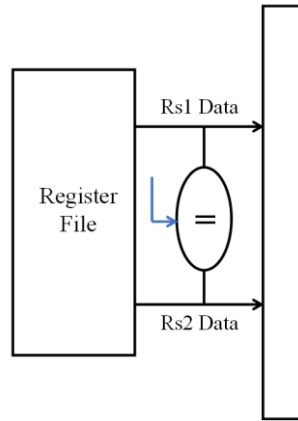


图3.7 拟将跳转判定提前到ID阶段示意图

**控制逻辑重构策略：**针对控制模块多层 if-else 结构导致的时序问题，建议实施三重优化：将 ALUOp 信号转换为独热编码减少译码层级；采用 Verilog 优先级 case 语句替代嵌套条件判断；创新设计两级流水化控制架构。

**数据依赖解决方案：**为缓解数据相关性问题，计划建立增强型前递网络：新增 EX→EX 直达通道解决连续算术指令依赖；设计 MEM→WB 专用旁路消除 Load-Use 型停顿；配置智能冲突检测器动态管理流水线冲突。

## 4 按键中断、计数器中断设计方案

### 4.1 中断方案设计

本系统采用二级中断处理架构，其中一级中断是基本的按键和计数器中断响应机制，二级中断专门用于处理中断嵌套的特殊情况。该方案的核心特征是所有中断类型统一跳转到固定入口地址32'h0000019c进行处理，简化了硬件实现但牺牲了灵活性。在中断返回机制上，设计为直接恢复至被中断的主程序流，不设置专门的中断返回层级，简化了控制逻辑但限制了多级中断支持。目前按键中断被整体赋予高于计数器中断的优先级，确保用户操作能够优先得到响应，但五个按键之间尚未实现差异化优先级，若按键2和按键3同时触发会随机响应其中之一。

实际运行流程为：当按键产生上升沿或下降沿时，触发一级中断处理流程；当计数器的值从低于阈值变为等于阈值时，触发计数器中断；如果当前正在处理计数器中断时发生按键中断，系统会将当前中断现场临时保存，优先响应按键操作后再恢复计数器中断处理。该方案在保证基本功能的前提下进行了适度简化，后续可考虑增加中断优先级配置寄存器，通过软件为不同按键分配不同的优先级别，例如将按钮0设为最高优先级，按钮4设为最低优先级。也可将统一中断入口改造为向量化中断系统，为每种中断源分配独立的入口地址，并增加中断状态寄存器(ISR)记录详细的中断上下文。

### 4.2 中断发生器 Int-controller 设计

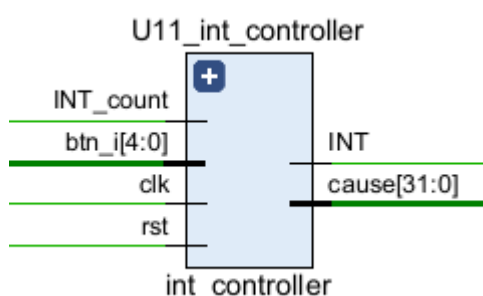


图4.1 中断发生器接口示意图

中断发生器(如图4.1所示)中断系统的前端检测单元，负责准确捕捉按键操作和计数器溢出事件。在硬件实现上，对输入的5位按键信号采用四级寄存器级联设计，通过移位寄存器{btn\_last, btn\_current}的形式实现对异步信号的同步

化处理，避免亚稳态问题。计数器输入同样经过两级寄存器进行时钟域同步，确保计数值采样稳定可靠。边沿检测单元采用差分比较机制，通过实时比较当前按键状态与保存的历史状态，生成上升沿和下降沿检测信号。当检测到任一按键状态变化时，立即将中断信号INT置为高电平，并将具体的中断原因编码到cause寄存器的低五位，如按键0的上升沿编码为6'b000001。

计数器中断检测使用阈值比较策略，设置COUNT\_THRESHOLD为特定门限值，当计数器的值从低于该阈值跳变至等于该阈值时，将INT信号置高并将cause寄存器的最高位(第5位)设为1。抗干扰机制采用多级信号稳定检测策略，按键信号需要经过三个时钟周期的采样稳定期，有效滤除宽度小于50ns的抖动干扰脉冲。所有关键寄存器包含全局复位控制，系统复位时中断信号和原因寄存器自动清零。

### 4.3 中断裁决器 Int\_judger 设计

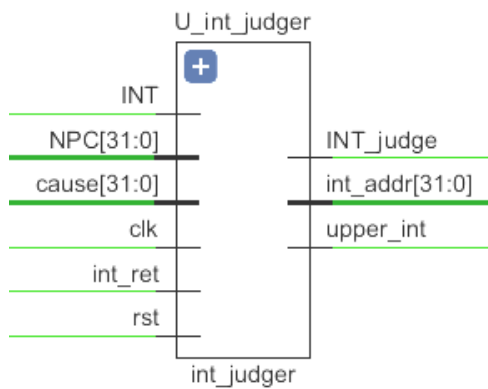


图4.2 中断裁决器接口示意图

中断裁决器(如图4.2所示)是中断处理流程的核心控制单元，实现了中断优先级判断和执行流管理。基本响应逻辑是在检测到INT信号有效且系统不处于中断处理状态时，立即将当前程序计数器值加4（指向中断后的下条指令）存入EPC寄存器，并将程序执行流跳转到固定中断入口地址32'h0000019c。嵌套中断处理机制主要针对计数器中断执行过程中的按键中断：当int\_handling标志有效表明正在处理计数器中断时，若检测到按键中断，系统将当前中断点地址存入嵌套专用寄存器nested\_epc，保持中断处理状态标志，并重新触发中断入口跳转。

中断返回流程通过识别ERET指令实现，若未设置嵌套标志，则清除int\_handling状态并跳转回EPC存储的主程序地址；若存在嵌套标志，则恢复

nested\_epc存储的地址继续执行被中断的中断程序，同时清除嵌套标志。优先级管理采用静态编码机制，按键中断优先级固定高于计数器中断，但所有按键具有相同优先级。

## 5 附加功能：Pblock 划分避免亚稳态

### 5.1 引入原因

在 FPGA 的设计实践中，亚稳态（Metastability）问题一直是影响可靠性的重要因素。尤其是在跨时钟域设计（CDC）中，不同的逻辑模块可能运行于不同的时钟频率或者时钟相位，如果信号在一个时钟周期内无法稳定，便容易引发亚稳态，导致系统行为不可预期。在复杂的 SoC 或 FPGA 设计中，跨模块的数据通信、时钟不对齐以及布线延迟的不可控，都会增加亚稳态出现的概率。

为此，我们引入 Pblock（逻辑区域约束）划分机制，将特定模块限定在特定的物理区域内布局，从而控制布线长度、降低延迟差异，提升跨模块同步的稳定性。这在处理高频时钟、异步FIFO、控制信号等敏感路径时尤其重要，能够有效减缓或者规避因后端布局引发的时序问题和亚稳态风险。

### 5.2 Pblock 实现方法

我和郑慧琳、谢巧、柯昀志三位同学合作完成这部分内容。

**选择关键模块：**首先分析 RTL 层级和综合报告，确定跨时钟域通信、时序敏感路径或高频控制模块。我们重点关注 FIFO 接口模块和控制逻辑模块，它们在设计中存在潜在的 CDC 问题。

**创建 Pblock：**在 Vivado GUI 中，使用 Floorplanning > Create Pblock 命令手动选择区域，或在 XDC 文件中添加约束：

```
create_pblock pblock_fifo_ctrl  
resize_pblock pblock_fifo_ctrl -add {SLICE_X0Y0:SLICE_X15Y50}  
add_cells_to_pblock [get_pblocks pblock_fifo_ctrl] [get_cells  
-hierarchical -filter {NAME =~ "*fifo_ctrl*"}]
```

**绑定时钟区域：**保证 Pblock 落在单一或连续的时钟区域内，避免跨多个 clock region 引入时钟树不均导致的问题。

**综合与实现检查：**使用 Vivado 的 Report Timing Summary 与 Report CDC 工具，对 Pblock 区域的模块重新分析，确保跨时钟信号的同步器逻辑被正确保留，且无新的 Setup/Hold 违例产生。

5.3 效果展示

在未引入 Pblock 之前，我们发现某些跨模块控制信号布线路径长，导致时序分析中存在若干 hold violation 与 max delay 报警。同时，由于布局不确定，多个关键控制模块可能被 Vivado 放在不同的逻辑区域，使得时钟偏移和布线干扰不可预测，具有较大亚稳态风险。

引入 Pblock 后，布线收敛更为集中，路径延迟被压缩在可控范围内，以下为前后对比结果(如表5.1所示)：

| 表 5.1 对比结果图        |  |               |           |
|--------------------|--|---------------|-----------|
| 项目                 |  | 无 Pblock      | 引入 Pblock |
| Hold Violations 数量 |  | 3 个（跨区域模块）    | 0 个       |
| 布局区域分布             |  | 离散，跨多个 Pblock | 区域集中      |
| 亚稳态概率              |  | 存在潜在时序漏洞      | 明显下降      |

此外，我们通过 Vivado 的 GUI 工具进行 Layout 可视化(如图5.1所示)，从图中可以看出模块被集中放置在一侧，布线较为规整，有助于时序工程优化与 CDC 分析。

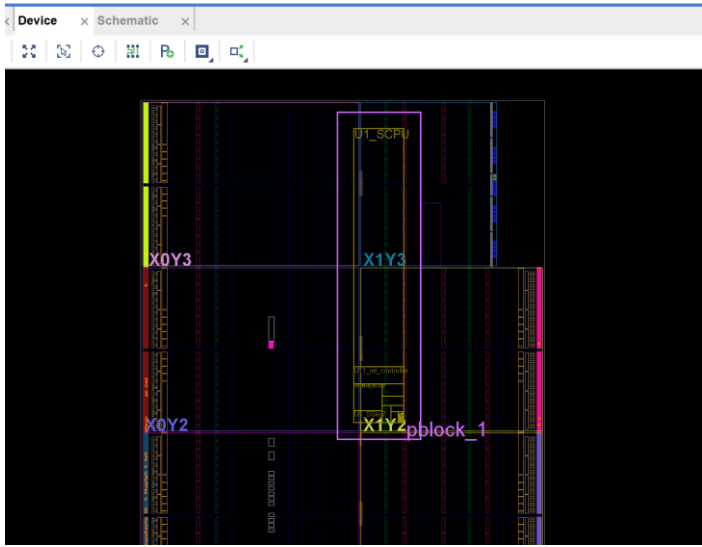


图5.1 layout结果图

综上所述，Pblock 技术不仅提升了布局结构的可控性，也为跨时钟通信系统提供了物理隔离基础，显著降低亚稳态发生的概率，为系统的稳定运行提供了保障。

## 6 实验测试方案及结果

### 6.1 测试方案

定义了sw\_i[2]为快慢速的控制拨码，sw\_i[5-7]为模式选择按钮（111代表查看地址流，110代表查看数据流，即寄存器x30）。

确定了测试代码，包括了完整的37条指令，特意把跳转指令放到的代码段的最后方，这样方便演示。

验收时仅需要简单的将地址流从头到尾演示一遍，其中可以实现中断处理。再演示数据流，对应预期结果一一查看，其中也可以实现中断处理。

### 6.2 测试代码与结果预期

测试代码及预期结果如表6.1：

表 6.1 测试代码及预期结果对应表

| 机器码        | 汇编指令              | 数据寄存器 x30  |
|------------|-------------------|------------|
| 0x123450B7 | lui x1, 0x12345   | 0x00000000 |
| 0x00008F33 | add x30, x1, x0   | 0x12345000 |
| 0x01000117 | auipc x2, 0x1000  | 0x12345000 |
| 0x00010F33 | add x30, x2, x0   | 0x01000008 |
| 0x06400193 | addi x3, x0, 100  | 0x01000008 |
| 0x00018F33 | add x30, x3, x0   | 0x00000064 |
| 0x0FF06213 | ori x4, x0, 0xFF  | 0x00000064 |
| 0x00020F33 | add x30, x4, x0   | 0x000000FF |
| 0x00F24293 | xori x5, x4, 0x0F | 0x000000FF |
| 0x00028F33 | add x30, x5, x0   | 0x000000F0 |
| 0x0F027313 | andi x6, x4, 0xF0 | 0x000000F0 |
| 0x00030F33 | add x30, x6, x0   | 0x000000F0 |
| 0x00219393 | slli x7, x3, 2    | 0x000000F0 |
| 0x00038F33 | add x30, x7, x0   | 0x00000190 |
| 0x0013D413 | srli x8, x7, 1    | 0x00000190 |
| 0x00040F33 | add x30, x8, x0   | 0x000000C8 |
| 0x40145493 | srai x9, x8, 1    | 0x000000C8 |
| 0x00048F33 | add x30, x9, x0   | 0x00000064 |
| 0x00200513 | addi x10, x0, 2   | 0x00000064 |
| 0x00A195B3 | sll x11, x3, x10  | 0x00000064 |
| 0x00058F33 | add x30, x11, x0  | 0x00000190 |
| 0x00A5D633 | srl x12, x11, x10 | 0x00000190 |
| 0x00060F33 | add x30, x12, x0  | 0x00000064 |
| 0x40A5D6B3 | sra x13, x11, x10 | 0x00000064 |
| 0x00068F33 | add x30, x13, x0  | 0x00000064 |

---

|            |                        |             |
|------------|------------------------|-------------|
| 0x00418733 | add x14, x3, x4        | 0x00000064  |
| 0x00070F33 | add x30, x14, x0       | 0x00000163  |
| 0x403207B3 | sub x15, x4, x3        | 0x00000163  |
| 0x00078F33 | add x30, x15, x0       | 0x0000009B  |
| 0x0041F833 | and x16, x3, x4        | 0x0000009B  |
| 0x00080F33 | add x30, x16, x0       | 0x00000064  |
| 0x0041E8B3 | or x17, x3, x4         | 0x00000064  |
| 0x00088F33 | add x30, x17, x0       | 0x000000FF  |
| 0x0041C933 | xor x18, x3, x4        | 0x000000FF  |
| 0x00090F33 | add x30, x18, x0       | 0x0000009B  |
| 0x0041A9B3 | slt x19, x3, x4        | 0x0000009B  |
| 0x00098F33 | add x30, x19, x0       | 0x00000001  |
| 0x0041BA33 | sltu x20, x3, x4       | 0x00000001  |
| 0x000A0F33 | add x30, x20, x0       | 0x00000001  |
| 0x0C81AA93 | slti x21, x3, 200      | 0x00000001  |
| 0x000A8F33 | add x30, x21, x0       | 0x00000001  |
| 0x0321BB13 | sltiu x22, x3, 50      | 0x00000001  |
| 0x000B0F33 | add x30, x22, x0       | 0x00000000  |
| 0x20000B93 | addi x23, x0, 0x200    | 0x00000000  |
| 0x000B8F33 | add x30, x23, x0       | 0x00000200  |
| 0x003BA023 | sw x3, 0(x23)          | 0x00000200  |
| 0x00018F33 | add x30, x3, x0        | 0x00000064  |
| 0x004B9223 | sh x4, 4(x23)          | 0x00000064  |
| 0x00020F33 | add x30, x4, x0        | 0x000000FF  |
| 0x005B8423 | sb x5, 8(x23)          | 0x000000FF  |
| 0x00028F33 | add x30, x5, x0        | 0x000000F0  |
| 0x000BAC03 | lw x24, 0(x23)         | 0x000000F0  |
| 0x000C0F33 | add x30, x24, x0       | 0x00000064  |
| 0x004B9C83 | lh x25, 4(x23)         | 0x00000064  |
| 0x000C8F33 | add x30, x25, x0       | 0x000000FF  |
| 0x004BDD03 | lhu x26, 4(x23)        | 0x000000FF  |
| 0x000D0F33 | add x30, x26, x0       | 0x000000FF  |
| 0x008B8D83 | lb x27, 8(x23)         | 0x000000FF  |
| 0x000D8F33 | add x30, x27, x0       | 0xFFFFFFFF0 |
| 0x008BCE03 | lbu x28, 8(x23)        | 0xFFFFFFFF0 |
| 0x000E0F33 | add x30, x28, x0       | 0x000000F0  |
| 0x01818263 | beq x3, x24, equal_br  | 0x000000F0  |
| 0x01818F33 | add x30, x3, x24       | 0x000000C8  |
| 0x00419263 | bne x3, x4, not_eq_br  | 0x000000C8  |
| 0x00418F33 | add x30, x3, x4        | 0x00000163  |
| 0x0041C263 | blt x3, x4, less_br    | 0x00000163  |
| 0x40320F33 | sub x30, x4, x3        | 0x0000009B  |
| 0x00325263 | bge x4, x3, greater_br | 0x0000009B  |
| 0x00320F33 | add x30, x4, x3        | 0x00000163  |
| 0x0041E263 | bltu x3, x4, less_u_br | 0x00000163  |
| 0x0041EF33 | or x30, x3, x4         | 0x000000FF  |

---

|            |                         |            |
|------------|-------------------------|------------|
| 0x00327263 | bgeu x4, x3, great_u_br | 0x000000FF |
| 0x01400EEF | jal x29, jump_target    | 0x000000FF |
| 0x000E8F33 | jalr x0, x29, 0         | 0x000000FF |
| 0x14800F93 | add x30, x29, x0        | 0x0000013C |
| 0x000F8067 | addi x31, x0, 0x148     | 0x0000013C |
| 0x0000006F | jalr x0, x31, 0         | 0x0000013C |
| 0x000E8067 | jal x0, infinite_loop   | 0x0000013C |

### 6.3 测试结果与评价

上机测试完整的实现了上述流程,实现了全部37条指令流水线及中断嵌套处理。我在验收文件夹中提供了完整的实现视频,下面仅用图6.1展示中断嵌套的x30结果。



图6.1 中断嵌套的x30结果

## 7 结论

本次实践基于RISC-V架构从零构建了一个支持中断的五级流水线CPU系统，在模块划分、接口设计、时序优化等方面进行了系统性的工程实现。特别是在中断处理机制和Pblock区域划分的附加功能中，不仅提升了系统的实用性与可靠性，也体现了工程实践中对系统稳定性和可扩展性的深入思考。

通过在Vivado平台上的调试与实现，成功验证了全部37条指令的正确执行，并实现了多源中断的裁决与响应。Pblock逻辑区域约束策略有效降低了因物理布局随机性导致的时序违例与亚稳态风险，展示了底层物理优化对于高可靠性系统设计的重要意义。

本项目为后续进行更复杂的SoC设计（如缓存管理、多核架构、操作系统接口等）打下了坚实基础，充分展现了硬件设计与系统思维的融合能力。

## 参考文献

- [1] 刘畅, 武延军, 吴敬征, 等. RISC-V 指令集架构研究综述[J]. 软件学报, 2021, 32(12): 3992-4024. DOI:10.13328/j.cnki.jos.006490.
- [2] 孙卫真, 刘雪松, 朱威浦, 等. 基于 RISC-V 的计算机系统综合实验设计[J]. 计算机工程与设计, 2021, 42(04): 1159-1165. DOI:10.16208/j.issn1000-7024.2021.04.037.
- [3] 陈志广, 刘皓铎, 卢宇彤. 基于 RISC-V 的计算机组成原理实验教学改革与实践[J]. 计算机教育, 2023, (02): 128-132. DOI:10.16512/j.cnki.jsjy.2023.02.033.
- [4] David Patterson, John L. Hennessy. 计算机组成与设计硬件/软件接口[M]. 第5版. 北京: 机械工业出版社, 2020年.
- [5] 刘畅, 杨毅, 李昊儒, 等. 处理器分支预测攻击研究综述[J]. 计算机学报, 2022, 45(12): 2475-2509.

## 致 谢

感谢熊小兵老师对实验方案、实验过程的规划和解释，一次性将实验的完整逻辑及评分标准进行说明有利于快速实现方案设计和代码实现。感谢郑慧琳同学、谢巧同学、柯昀志同学和我在方案设计上的讨论，集思广益帮助我们规避了很多不必要的麻烦，我们同时以合作的方式实现了附加功能PBlock划分，让我们的代码效率得到了很大提升。最后，感谢自己在项目高效完成上起到的决定性作用。

## 附录：模块核心代码列举

### A. SCPU模块核心代码

```
module SCPU(  
    input      clk,          // clock  
    input      reset,        // reset  
    input MIO_ready,  
    input INT,  
    input [31:0] cause,  
    input [31:0] inst_in,    // instruction  
    input [31:0] Data_in,    // data from data memory  
  
    output mem_w,            // output: memory write signal  
    output [31:0] PC_out,    // PC address  
  
    output [31:0] Addr_out,  // ALU output  
    output [31:0] Data_out, // data to data memory  
    output [2:0] dm_ctrl,  
    output CPU_MIO  
  
);  
.....  
irq_judge_unit U_irq_judge(  
    .clk_main(clk),  
    .rst_n(reset),  
    .irq_in(INT),  
    .cause_in(cause),  
    .npc_in(NPC),  
    .irq_ret(EX_MEM_int_ret),  
    .irq_flag(INT_judge),  
    .cause_flag(cause_judge),  
    .irq_addr(int_addr),  
    .irq_nested(upper_int)  
);  
  
PipeRegArray IF_ID_PipeRegArray(  
    .clk(clk),  
    .rst(reset),  
    .writeEn(IF_ID_write_enable),  
    .flushEn(IF_ID_flush),  
    .saveOut(INT_judge),  
    .upperInt(upper_int),
```

```

        .loadOut(EX_MEM_int_ret),
        .dataIn(IF_ID_in),
        .dataOut(IF_ID_out)
    );

```

```

PipeRegArray ID_EX_PipeRegArray(
    .clk(clk),
    .rst(reset),
    .writeEn(ID_EX_write_enable),
    .flushEn(ID_EX_flush),
    .saveOut(INT_judge),
    .upperInt(upper_int),
    .loadOut(EX_MEM_int_ret),
    .dataIn(ID_EX_in),
    .dataOut(ID_EX_out)
);

```

```

PipeRegArray EX_MEM_PipeRegArray(
    .clk(clk),
    .rst(reset),
    .writeEn(EX_MEM_write_enable),
    .flushEn(EX_MEM_flush),
    .saveOut(INT_judge),
    .upperInt(upper_int),
    .loadOut(EX_MEM_int_ret),
    .dataIn(EX_MEM_in),
    .dataOut(EX_MEM_out)
);

```

```

PipeRegArray MEM_WB_PipeRegArray(
    .clk(clk),
    .rst(reset),
    .writeEn(MEM_WB_write_enable),
    .flushEn(MEM_WB_flush),
    .saveOut(INT_judge),
    .upperInt(upper_int),
    .loadOut(EX_MEM_int_ret),
    .dataIn(MEM_WB_in),
    .dataOut(MEM_WB_out)
);

```

```

BranchFlushDetect U_BranchFlushDetect(
    .npcOpIn(ID_EX_NPCOp),
    .isZero(Zero),

```

```

        .intSignal(int_sig),
        .npcOpOut(npcOpOut),
        .ifIdFlush(ifIdFlushBranch),
        .idExFlush(idExFlushBranch)
        // .exMemFlush(exMemFlushBranch)
    );

    HazardStall U_HazardStall(
        .ifIdRs1(IF_ID_rs1),
        .ifIdRs2(IF_ID_rs2),
        .idExRd(ID_EX_rd),
        .idExWdSel(ID_EX_WDSel),
        .stallFlag(stallFlag),
        .ifIdWriteEnable(ifIdWriteEnableStall),
        .idExFlush(idExFlushStall)
    );

    DataForwardUnit U_DataForwardUnit(
        .exMemRegWrite(EX_MEM_RegWrite),
        .exMemRd(EX_MEM_rd),
        .memWbRegWrite(MEM_WB_RegWrite),
        .memWbRd(MEM_WB_rd),
        .idExRs1(ID_EX_rs1),
        .idExRs2(ID_EX_rs2),
        .forwardA(ForwardA),
        .forwardB(ForwardB)
    );

    assign IF_ID_flush = ifIdFlushBranch;
    assign ID_EX_flush = idExFlushBranch | idExFlushStall;
    assign EX_MEM_flush=0;
    assign MEM_WB_flush=0;
    assign IF_ID_write_enable = ifIdFlushBranch | ifIdWriteEnableStall;
    assign ID_EX_write_enable=1;
    assign EX_MEM_write_enable=1;
    assign MEM_WB_write_enable=1;

    // instantiation of pc unit
    PC U_PC(
        .clkIn(clk),
        .rstN(reset),
        .npcIn(NPC),
        .pcOut(PC_out)
    );

```

```

);
NPC U_NPC(
    .pcIn(PC_out),
    .npcOpSel(npcOpOut),
    .immVal(ID_EX_immout),
    .npcOut(NPC),
    .aluResult(aluout),
    .stallFlag(stallFlag),
    .pcOld(ID_EX_PC_out),
    .intSignal(int_sig),
    .intAddr(int_addr)
);

assign
IF_ID_in={int_ret,PC_out,rs1,rs2,rd,iimm_shamt,iimm,simm,bimm,uimm,jimm,Op,Funct7,Funct3};

assign IF_ID_int_ret=IF_ID_out[145:145];
assign IF_ID_PC_out=IF_ID_out[144:113];
assign IF_ID_rs1=IF_ID_out[112:108];
assign IF_ID_rs2=IF_ID_out[107:103];
assign IF_ID_rd=IF_ID_out[102:98];
assign IF_ID_iimm_shamt=IF_ID_out[97:93];
assign IF_ID_iimm=IF_ID_out[92:81];
assign IF_ID_simm=IF_ID_out[80:69];
assign IF_ID_bimm=IF_ID_out[68:57];
assign IF_ID_uimm=IF_ID_out[56:37];
assign IF_ID_jimm=IF_ID_out[36:17];
assign IF_ID_Op=IF_ID_out[16:10];
assign IF_ID_Funct7=IF_ID_out[9:3];
assign IF_ID_Funct3=IF_ID_out[2:0];

// instantiation of control unit
ctrl U_ctrl(
    .Op(IF_ID_Op), .Funct7(IF_ID_Funct7), .Funct3(IF_ID_Funct3),
    .RegWrite(RegWrite), .MemWrite(Memwrite),
    .EXTOp(EXTOp), .ALUOp(ALUOp), .NPCOp(NPCOp),
    .ALUSrc(ALUSrc), .GPRSel(GPRSel), .WDSel(WDSel), .dm_ctrl(dm_c), .Memread(Memread)
);

ImmExtender U_ImmExtender(
    .shamtIn(IF_ID_iimm_shamt), .iImm(IF_ID_iimm), .sImm(IF_ID_simm), .bImm(IF_ID_bimm),
    .uImm(IF_ID_uimm), .jImm(IF_ID_jimm),

```

```

        .extOp(EXTOp), .immOut(immout)
    );
    RF U_RF(
        .clk(clk), .rst(reset), .save_out(INT_judge),
        .load_out(EX_MEM_int_ret),
        .RFWr(MEM_WB_RegWrite),
        .A1(IF_ID_rs1), .A2(IF_ID_rs2), .A3(MEM_WB_rd),
        .WD(WD),
        .RD1(RD1), .RD2(RD2)
        // .reg_sel(reg_sel),
        // .reg_data(reg_data)
    );

    assign
ID_EX_in={IF_ID_int_ret,IF_ID_rd,IF_ID_rs1,IF_ID_rs2,RegWrite,Memread,Memwrite,EXTO
p,ALUOp,NPCOp,ALUSrc,WDSel,dm_c,immout,RD1,RD2,IF_ID_PC_out};

    assign ID_EX_int_ret=ID_EX_out[167:167];
    assign ID_EX_rd=ID_EX_out[166:162];
    assign ID_EX_rs1=ID_EX_out[161:157];
    assign ID_EX_rs2=ID_EX_out[156:152];
    assign ID_EX_RegWrite=ID_EX_out[151:151];
    assign ID_EX_Memread=ID_EX_out[150:150]; // not need
    assign ID_EX_Memwrite=ID_EX_out[149:149];
    assign ID_EX_EXTOp=ID_EX_out[148:143];
    assign ID_EX_ALUOp=ID_EX_out[142:138];
    assign ID_EX_NPCOp=ID_EX_out[137:135];
    assign ID_EX_ALUSrc=ID_EX_out[134:134];
    assign ID_EX_WDSel=ID_EX_out[133:131];
    assign ID_EX_dm_c=ID_EX_out[130:128];
    assign ID_EX_immout=ID_EX_out[127:96];
    assign ID_EX_RD1=ID_EX_out[95:64];
    assign ID_EX_RD2=ID_EX_out[63:32];
    assign ID_EX_PC_out=ID_EX_out[31:0];

    wire [31:0] A_mux_out, B_mux_in_out;

    mux3 #(32) u_muxA(
        .sel(ForwardA),
        .d0(ID_EX_RD1),
        .d1(WD),
        .d2(EX_MEM_aluout),
        .y(A_mux_out)
    );

```

```

mux3 #(32) u_muxB(
    .sel(ForwardB),
    .d0(ID_EX_RD2),
    .d1(WD),
    .d2(EX_MEM_aluout),
    .y(B_mux_in_out)
);
assign A = A_mux_out;
assign B_in = B_mux_in_out;

assign B = (ID_EX_ALUSrc) ? ID_EX_immout : B_in;

// instantiation of alu unit
alu
U_alu(.A(A), .B(B_in), .ALUOp(ID_EX_ALUOp), .C(aluout), .Zero(Zero), .PC(ID_EX_PC_out));

assign
EX_MEM_in={ID_EX_int_ret,aluout,ID_EX_PC_out,ID_EX_WDsel,ID_EX_RegWrite,ID_EX_Memread,
ID_EX_Memwrite,ID_EX_NPCOp,ID_EX_dm_c,ID_EX_rd,B_in};

assign EX_MEM_int_ret = EX_MEM_out[113:113];
assign EX_MEM_aluout = EX_MEM_out[112:81];
assign EX_MEM_PC_out = EX_MEM_out[80:49];
assign EX_MEM_WDsel = EX_MEM_out[48:46];
assign EX_MEM_RegWrite = EX_MEM_out[45:45];
assign EX_MEM_Memread = EX_MEM_out[44:44];
assign EX_MEM_Memwrite = EX_MEM_out[43:43];
assign EX_MEM_NPCOp = EX_MEM_out[42:40]; // not need
assign EX_MEM_dm_c = EX_MEM_out[39:37];
assign EX_MEM_rd = EX_MEM_out[36:32];
assign EX_MEM_RD2 = EX_MEM_out[31:0];

assign Addr_out = EX_MEM_aluout; // EX_MEM
assign Data_out = EX_MEM_RD2; // EX_MEM
assign mem_w = EX_MEM_Memwrite;
assign dm_ctrl = EX_MEM_dm_c;

assign
MEM_WB_in={EX_MEM_int_ret,Data_in,EX_MEM_RegWrite,EX_MEM_rd,EX_MEM_WDsel,EX_MEM_aluout,EX_MEM_PC_out};

assign MEM_WB_int_ret = MEM_WB_out[105:105];
assign MEM_WB_Data_in = MEM_WB_out[104:73];

```

```

assign MEM_WB_RegWrite = MEM_WB_out[72:72];
assign MEM_WB_rd = MEM_WB_out[71:67];
assign MEM_WB_WDSEL = MEM_WB_out[66:64];
assign MEM_WB_aluout = MEM_WB_out[63:32];
assign MEM_WB_PC_out = MEM_WB_out[31:0];

always @*
begin
    case (MEM_WB_WDSEL)
        `WdSelFromAlu: WD <= MEM_WB_aluout;
        `WdSelFromMem: WD <= MEM_WB_Data_in;
        `WdSelFromPc: WD <= MEM_WB_PC_out + 4;
    endcase
end

endmodule

```

## B. ImmExtender模块核心代码

```

module ImmExtender(
    input [4:0] shamtIn,
    input [11:0] iImm, //instr[31:20], 12 bits
    input [11:0] sImm, //instr[31:25, 11:7], 12 bits
    input [11:0] bImm, //instrD[31], instrD[7], instrD[30:25], instrD[11:8],
12 bits
    input [19:0] uImm,
    input [19:0] jImm,
    input [5:0] extOp,

    output reg [31:0] immOut);

always @(*)
    case (extOp)
        `ExtCtrlItypeShamt: immOut<={27'b0,shamtIn[4:0]};
        `ExtCtrlItype: immOut <= {{{32-12}{iImm[11]}}, iImm[11:0]};
        `ExtCtrlStype: immOut <= {{{32-12}{sImm[11]}}, sImm[11:0]};
        `ExtCtrlBtype: immOut <= {{{32-13}{bImm[11]}}, bImm[11:0], 1'b0};
        `ExtCtrlUtype: immOut <= {uImm[19:0], 12'b0}; //??????????12??0
        `ExtCtrlJtype: immOut <= {{{32-21}{jImm[19]}}, jImm[19:0], 1'b0};
        default:
            immOut <= 32'b0;
    endcase

endmodule

```

### C. DataForwardUnit模块核心代码

```
module DataForwardUnit(
    input exMemRegWrite,
    input [4:0] exMemRd,
    input memWbRegWrite,
    input [4:0] memWbRd,
    input [4:0] idExRs1,
    input [4:0] idExRs2,
    output reg [1:0] forwardA,
    output reg [1:0] forwardB
);

always @*
begin
    if(exMemRegWrite && exMemRd == idExRs1 && exMemRd!=0)
        forwardA = 2'b10;
    else if(memWbRegWrite && memWbRd == idExRs1 && memWbRd!=0)
        forwardA = 2'b01;
    else
        forwardA = 2'b00;
end

always @*
begin
    if(exMemRegWrite && exMemRd == idExRs2 && exMemRd!=0)
        forwardB = 2'b10;
    else if(memWbRegWrite && memWbRd == idExRs2 && memWbRd!=0)
        forwardB = 2'b01;
    else
        forwardB = 2'b00;
end

endmodule
```

### D. PipeRegArray模块核心代码

```
module PipeRegArray(
    input clk,
    input rst,
    input writeEn,
    input flushEn,
    input saveOut,
    input upperInt,
    input loadOut,
    input [255:0] dataIn,
```

```

output reg [255:0] dataOut
);

reg [255:0] outInt;

always @ (posedge clk or posedge rst)
begin
    if(rst)
    begin
        dataOut=0;
    end
    else
    begin
        if(writeEn)
        begin
            if(saveOut)
            begin
                if(!upperInt) outInt<=dataOut;
                dataOut<=0;
            end
            else if(flushEn)
                dataOut<=0;
            else if(loadOut)
                dataOut<=outInt;
            else
                dataOut<=dataIn;
        end
    end
end

endmodule

```

## E. HazardStall模块核心代码

```

module HazardStall(
    input [4:0] ifIdRs1,
    input [4:0] ifIdRs2,
    input [4:0] idExRd,
    input [2:0] idExWdSel,
    output reg stallFlag,
    output reg ifIdWriteEnable,
    output reg idExFlush
);

always @*
begin

```

```

        if((idExRd!=0 && (ifIdRs1==idExRd || ifIdRs2==idExRd) && idExWdSel==3'b001))
        begin
            stallFlag<=1;
            ifIdWriteEnable<=0;
            idExFlush<=1;
        end
    else
    begin
        stallFlag<=0;
        ifIdWriteEnable<=1;
        idExFlush<=0;
    end
end
endmodule

```

## F. NPC模块核心代码

```

module NPC(
    pcIn, npcOpSel, immVal, npcOut, aluResult, stallFlag, pcOld, intSignal, intAddr
); // next pc module
    input [31:0] pcIn;           // pc
    input [2:0] npcOpSel;       // next pc operation
    input [31:0] immVal;       // immediate
    input [31:0] aluResult;
    output reg [31:0] npcOut;   // next pc
    input stallFlag;
    input [31:0] pcOld;
    input intSignal;
    input [31:0] intAddr;

    wire [2:0] npcOpMux;
    wire [31:0] pcPlus4;

    assign npcOpMux = intSignal ? `NIntAddr : npcOpSel;
    assign pcPlus4 = stallFlag ? pcIn : pcIn + 4; // pc + 4

    always @(*) begin
        case (npcOpMux)
            `NPlus4: npcOut = pcPlus4;
            `NBranch: npcOut = pcOld + immVal;
            `NJump: npcOut = pcOld + immVal;
            `NJalr: npcOut = aluResult;
            `NIntAddr: npcOut = intAddr;
            default: npcOut = pcPlus4;
        endcase
    end
endmodule

```

```
end // end always
```

```
endmodule
```

## G. PC模块核心代码

```
module PC(  
    input clkIn,  
    input rstN,  
    input [31:0] npcIn,  
    output reg [31:0] pcOut  
);  
    always @(posedge clkIn or posedge rstN) begin  
        if (rstN)  
            pcOut <= 32'b0;  
        else  
            pcOut <= npcIn;  
        end  
    endmodule
```